

WORKATO GUIDE

Multi-Channel Delivery Router – Workato Implementation Guide



Workflow ID: 22-multi-channel-delivery-router • Backstory public rollout asset

This file is a plain-English implementation guide. It is not an importable JSON artifact.

Workato moves reusable automation between workspaces as recipe packages, and package import uses a `.zip` file in Recipe Lifecycle Management. Imported connections arrive as placeholders and must be authenticated in the destination workspace. Build this workflow in Workato as recipes, recipe functions, connectors, properties, and tables, then export/import the package when you need to promote it between environments.

Workflow Summary

- Workflow ID: `22-multi-channel-delivery-router`
- Category: platform-enablement
- Source trigger in this catalog: Webhook / Sub-workflow |
- Recommended Workato trigger surface: Callable recipe or Recipe Function invocation from a parent orchestrator
- Delivery targets: Slack channel or direct message, Microsoft Teams channel or chat, Email delivery, Webhook, queue, database, or downstream workflow sink

What To Create In Workato

- Slack connector or Workbot for Slack for channel and DM delivery
- Microsoft Teams connector or Workbot for Microsoft Teams
- SMTP-compatible email connector for outbound delivery
- Lookup tables, data tables, or project properties for routing and canonical maps
- Webhook connector, database connector, queue adapter, or downstream orchestrator action

Build Order

1. Create or choose the target project and folder where this workflow will live.
2. Confirm the native source-system connectors needed by this workflow are installed and authenticated.

3. Create the Recipe Function `normalize_run_context` so reusable logic is not trapped inside one large recipe.
4. Create the Recipe Function `load_source_records` so reusable logic is not trapped inside one large recipe.
5. Create the Recipe Function `assemble_business_context` so reusable logic is not trapped inside one large recipe.
6. Create the Recipe Function `render_delivery_payload` so reusable logic is not trapped inside one large recipe.
7. Create the Recipe Function `record_run_summary` so reusable logic is not trapped inside one large recipe.
8. Build the primary recipe `22_multi_channel_delivery_router_primary_recipe` with the trigger surface Callable recipe or Recipe Function invocation from a parent orchestrator.
9. Store routing defaults, destination IDs, and mode switches in connections, environment properties, project properties, lookup tables, or data tables.
10. Test the recipe and each function with representative payloads, then export the project as a Workato package `.zip` if you need to move it to QA or production.

Recipe Functions To Create

- `normalize_run_context` : Normalize the Multi-Channel Delivery Router trigger into the shared `run_context` contract. Inputs: `trigger_payload`, workflow defaults. Outputs: `run_context`. Native features: Recipe function by Workato, Formula mode, Variables.
- `load_source_records` : Load and normalize the primary Multi-Channel Delivery Router business inputs into `source_record` objects. Inputs: `run_context`. Outputs: `source_record[]`. Native features: Recipe function by Workato, Lookup tables or data tables.
- `assemble_business_context` : Join the intermediate context needed for Multi-Channel Delivery Router, especially Resolve Destination. Inputs: `source_record[]`, `run_context`. Outputs: `source_record[]`, `enrichment_context` candidates. Native features: Lists, Object actions, Data tables.
- `render_delivery_payload` : Render normalized `delivery_payload` objects for Multi-Channel Delivery Router. Inputs: `structured_ai_json`, `source_record`, `run_context`. Outputs: `delivery_payload`. Native features: Recipe function by Workato, Object construction, Formula mode.
- `record_run_summary` : Capture deterministic summary, dedupe, and retry state for Multi-Channel Delivery Router. Inputs: `run_context`, delivery results. Outputs: execution summary. Native features: Variables, Data tables.

Primary Recipe Outline

1. Call `normalize_run_context`
2. Use native connector action for Resolve Destination
3. Use an LLM connector or AI step to return strict JSON for Adapt Message Format
4. Use database, queue, webhook, or downstream app connector for Deliver to Preferred Channel

5. Use Slack connector for Apply Fallback Route
6. Record deterministic run summary and retry state

Contracts To Preserve

- `run_context` : workflow-level execution context such as trigger, mode, lookback, delivery mode, and dry-run state.
- `source_record` : normalized business record from the source adapter or source-system action layer.
- `enrichment_context` : MCP or native app enrichment results used only during synthesis.
- `delivery_payload` : deterministic delivery fields for the final native connector or app action.

Structured AI Requirements

- `adapt_message_format` : return JSON only with keys `title` , `summary` , `confidence` , `recommended_actions` , `source_refs` . Intent: Transforms the same insight into Slack-safe markdown, Teams-safe copy/cards, email HTML/plain text, or webhook-safe JSON envelopes.

Validation Checklist

- No repeated Universal HTTP or custom-action sprawl for Slack, calendar, CRM, or email delivery when a native connector exists.
- All secrets live in connections, environment properties, or project properties rather than formula literals.
- LLM or agent output is validated as structured JSON before any delivery or persistence step runs.
- Recipe Functions, lookup tables, and data tables own reuse, routing, and dedupe instead of large inline scripts.
- If this build is moved between workspaces, export and import it as a Workato package `.zip` , then reconnect placeholder connections in the target workspace.

Official References

- [Recipe lifecycle management](#)
- [Importing packages](#)
- [Recipe function by Workato](#)
- [Using the Workato Connector SDK](#)
- [Slack connector](#)
- [Google Calendar connector](#)

